

PROJEKTDOKUMENTATION Entwicklung der Restaurantverwaltungssoftware „Ray Foodie“

Entwickler: Luis Reynaldo Cedeño Manzanilla

Datum: Juni 2026

1. Einleitung und Projektidee
2. Ziel des Projekts und funktionale Anforderungen
3. Architektur der Benutzeroberfläche (UI-Design)
4. Software-Architektur und Datenstruktur
5. Fehlerprotokoll und technische Lösungen
6. Verwendete Hilfsquellen und Methodik
7. Fazit und Ausblick

Einleitung und Projektidee

Im Rahmen dieses Softwareprojekts wurde eine maßgeschneiderte Desktop-Anwendung für Gastronomiebetriebe in der Programmiersprache C# (.NET) entwickelt. Die Kernidee hinter „Ray Foodie“ ist es, den oft fehleranfälligen und zeitaufwendigen Bestellvorgang in einem Restaurant massiv zu vereinfachen. Gleichzeitig lag der Fokus auf der Automatisierung der Rechnungsstellung, um dem Servicepersonal administrative Arbeit abzunehmen.

Ziel des Projekts

Das primäre Ziel des Projekts war der Nachweis, wie mit modernen C#-Bordmitteln eine benutzerfreundliche, performante und praxisnahe Restaurantsoftware realisiert werden kann. Dabei standen drei Säulen im Vordergrund:

1. **Intuitive Bedienung:** Das Personal muss ohne lange Einarbeitungszeit Bestellungen aufnehmen können.
2. **Echtzeit-Berechnung:** Fehlberechnungen bei den Gesamtsummen werden durch automatische Live-Kalkulationen ausgeschlossen.
3. **Professioneller Belegdruck:** Die Erstellung eines physischen oder digitalen Kassensbons direkt nach Abschluss des Bestellvorgangs.

Architektur der Benutzeroberfläche (UI-Design)

Strukturierung der Oberfläche

Die Anwendung verfügt über eine klare, dreigeteilte Benutzeroberfläche, um den kognitiven Aufwand für den Benutzer zu minimieren.

Das dynamische Produkt-Katalogsystem (FlowLayoutPanel)

Anstatt traditionelle Tabellen zu nutzen, wurde für den zentralen Bereich ein `FlowLayoutPanel` gewählt. Dieses Steuerelement ermöglicht es, benutzerdefinierte Steuerelemente (`UserControl`) dynamisch in Form von Produktkarten zu laden. Jede Karte repräsentiert ein Gericht mit `Bild`, `Titel`, `Beschreibung` und `Preis`. Das Panel wurde so konfiguriert (`AutoScroll = True`), dass bei einer großen Anzahl von Artikeln automatisch eine Bildlaufleiste erscheint, ohne das Layout zu zerstören.

Die Bestellübersicht (DataGridView)

Auf der rechten Seite befindet sich die Live-Bestellliste („Bestellübersicht“). Hier wurde ein `DataGridView` verwendet, das die ausgewählten Artikel strukturiert in den Spalten *Produkt*, *Menge*, *Einzelpreis* und *Gesamtpreis* auflistet.

Das Datenmodell (*Producto.cs*)

Die Logik der Anwendung basiert auf einer klaren objektorientierten Struktur. Die Klasse `Producto` dient als Datenmodell für alle angebotenen Speisen und Getränke. Sie kapselt folgende Eigenschaften:

- `Nombre` (`String`): Der Name des Artikels.
- `Precio` (`Double`): Der Preis in Euro.
- `Descripcion` (`String`): Kurze Details zum Gericht.
- `Categoria` (`String`): Filter-ID für das Menü.
- `Imagen` (`Image`): Visuelle Repräsentation des Produkts.

Das modulare UI-Element (*TarjetaPlatillo.cs*)

Dieses `UserControl` fungiert als visuelle Brücke. Es abonniert ein benutzerdefiniertes Event (`AlSeleccionarProducto`), das dem Hauptformular signalisiert, wenn ein Benutzer auf das Produkt klickt.

Das Druck-Subsystem

Für die Belegvorschau und den Druck wurden die nativen .NET-Komponenten `PrintDocument`, `PrintPreviewDialog` und `PrintDialog` implementiert. Der Bon wird über Koordinaten (`e.Graphics.DrawString`) pixelgenau gezeichnet.

Fehlerprotokoll und technische Lösungen

Während der Entwicklungsphase traten verschiedene technische Herausforderungen auf, die systematisch analysiert und behoben wurden:

1. WFO1000 Designer-Warnung

- a. *Problem:* Der Visual Studio-Designer unterbrach den Build-Vorgang mit Warnungen bezüglich der komplexen Bildeigenschaften im UserControl1.
- b. *Lösung:* Dekoration der Eigenschaften mit den Metadaten-Attributen `[Browsable(false)]` und `[DesignerSerializationVisibility(Hidden)]`.

2. Syntaxfehler durch Typografie (Leerzeichen)

- a. *Problem:* Ein unbemerktes Leerzeichen in einer Variablen-Deklaration (`int cant Actual`) führte zu Folgefehlern im gesamten Klick-Event.
- b. *Lösung:* Bereinigung des Quellcodes nach der *camelCase*-Konvention zu `cantActual`.

3. System.IO.FileNotFoundException bei Produktbildern

- a. *Problem:* Das Programm stürzte ab, weil absolute Pfade verwendet wurden und die Dateiendungen im Code (`.PNG`) nicht mit den realen Dateien (`.jpg`) übereinstimmten.
- b. *Lösung:* Verschiebung des Images-Ordners in das Verzeichnis `bin/Debug/net10.0-windows`, Nutzung von `Application.StartupPath` und Anpassung der Dateiendungen im Code.

4. Leere Druckvorschau („Das Dokument enthält keine Seiten“)

- a. *Problem:* Das `PrintPreviewDialog` zeigte ein leeres Dokument, da das `PrintPage`-Event im Designer-Lebenszyklus nicht rechtzeitig gebunden wurde.
- b. *Lösung:* Explizite manuelle Zuweisung des Dokuments zur Laufzeit direkt im Klick-Event des Buttons:
`printPreviewDialog1.Document = printDocument1;`

Verwendete Hilfsquellen

Die erfolgreiche Umsetzung des Projekts innerhalb des vorgegebenen Zeitrahmens wurde durch eine gezielte Nutzung moderner Informationsmedien und Entwicklungswerkzeuge unterstützt. Es wurden ausschließlich folgende Quellen zur Problemlösung und Wissenserweiterung herangezogen:

- **Gemini & ChatGPT (Künstliche Intelligenz):** Bildgenerator
- **StackOverflow:** Recherche-Plattform zur Lösung von Problemen mit Fenster-Ankern (Anchor- und Dock-Eigenschaften) sowie zur Behebung von UI-Blockaden.
- **Microsoft MSDN (Offizielle Dokumentation):** Nachschlagewerk für die native Grafikbibliothek `System.Drawing` und die Funktionsweise von Druckkomponenten in Windows Forms.
- **YouTube:** Visuelle Unterstützung beim Aufbau reaktiver Benutzeroberflächen und dem Design von modernen Seitenmenüs.

Fazit, Kassenbon-Struktur und Unterschrift

Struktur des generierten Kassenbons

Die gedruckte Rechnung entspricht allen Anforderungen der Projektbeschreibung und gibt folgende Daten aus:

- Name des Restaurants („Ray Foodie“)
- Dynamische Rechnungsnummer sowie Datum und Uhrzeit
- Tabellarische Auflistung: Produktname, Menge, Einzelpreis und Gruppen-Subtotal
- Steuernachweis (inkl. 19% MwSt.) und die Gesamtsumme
- Höfliche Abschluss-Dankesnachricht für den Kunden

Fazit

Das Projekt „Ray Foodie“ zeigt eindrucksvoll, dass C# Windows Forms eine robuste und hocheffiziente Plattform für die Entwicklung von lokalen Gastronomie-Anwendungen darstellt. Alle Meilensteine – von der dynamischen UI-Aktualisierung bis hin zum automatisierten Belegdruck – wurden erfolgreich und fehlerfrei umgesetzt.

Entwickler-Signatur:

Luis Reynaldo Cedeño Manzanilla

Ray Foodie